```
from django.db import migrations

class Migration(migrations.Migration):
    dependencies = [
        ("promoters", "0003_promoter_common_name_promoter_full_name"),
    ]

    operations = []
```

The migration framework automatically makes the new script dependent on the previous one.

A stub migration script starts with no operations.

With 0004_data_fullname.py in place, you now have somewhere to write the data translation code. This script gets run after the 0003 migration, so the full and common name fields will exist. To populate those fields, enumerate all the existing rows, copying the first and last name contents into the full and common name fields. This work goes in a function that can live in the migration file. The migration system supports moving forward and backward in table state, so it is best practice to write code that undoes the translation work as well.

> **NOTE** In some situations, you simply can't move backward through a migration. The transformation on your data may be one-way. If you do not provide a function for moving backward to `RunPython`, Django raises an exception if someone attempts to undo this migration step.

To write a Django view that enumerates all `Promoter` objects, import Promoter and run `Promoter.objects.all()`. Unfortunately, life in a migration script isn't that simple. Importing `Promoter` gives you the current state of the model. The state of `Promoter` in a migration script may be different from the current state of the model—that's the purpose of migration scripts. Consider the case where you are on your 10th change to `Promoter`, the 4th migration script needs the state of Promoter six changes ago.

Django can build any version of your model from the information inside the migration scripts. You get at the version corresponding to the migration script by calling `apps.get_model()`, where `apps` is one of the two arguments sent in to the function called by `RunPython`. The second argument is `schema_editor`, and although it isn't something normal humans use, it has to be in the function signature. The migration system in Django is pluggable, and you can replace it with your own code. Doing so requires building `SchemaEditor` objects, which are what the migration system is passing to your function call. I've never met anyone, outside the Django core team, who has ever needed to do this though.

Listing 10.7 contains the code that translates the first and last name fields into their corresponding full and common name versions. It contains two functions, `de_westernize_names()` and `re_westernize_names()`, for migrating forward and backward, respectively. References to these functions are passed to a `RunPython` object, which is included in the `Migration` class's `operations` list.

Listing 10.7 A migration script that calls Python to transform data during migration

```
# RiffMates/promoters/migrations/0004_data_fullname.py
from django.db import migrations
```